



Softwarearchitektur Projektarbeit

Projektplan

Gruppe 1 - Team 2

Ayleen Edenhauser, Matteo Volperino,
Jakob Oberhofer, Leonid Georg Mikhailovic Stommel (Ljonja)

WS 2025/26

Contents

1	Einleitung	3
1.1	Workflow	3
1.1.1	Sequenzdiagramm	5
1.1.2	Fachliches Klassendiagramm	6
1.2	Meilensteine	7
1.3	Issues	7
2	User	9
2.1	Attribute	9
2.1.1	Rollen	9
2.2	Involvierte Klassen	10
3	Produkte	11
3.1	Attribute	11
3.1.1	Bewertungen	11
3.2	Abonnements	12
3.2.1	Ablauf	13
3.3	Involvierte Klassen	13
4	Warenkorb	15
4.1	Attribute	15
4.2	Involvierte Klassen	16
5	Bestellungen	17
5.1	Attribute	17
5.1.1	Status	17
5.2	Zahlungsabwicklung	18
5.3	Erstellung einer Bestellung	18
5.4	Involvierte Klassen	18
6	Fehlerbehandlung	19

1 Einleitung

Dieser Projektplan beschreibt die geplante Umsetzung unseres Projekts. In der Einleitung wird zunächst die grundlegende Nutzung des Webshops erläutert. Die folgenden Kapitel behandeln die verschiedenen *Aspekte des Projekts* und zeigen auf, wie diese *miteinander interagieren*. Ziel dieses Dokuments ist es, einen **strukturierten und nachvollziehbaren Projektplan** zu erstellen, der als Orientierung für die weitere Projektarbeit dient.

1.1 Workflow

Ein User besucht den Webshop und landet auf der Homepage, welche eine Gallery View aller Produkte ist. Der User sieht zu jedem Produkt:

- Das **Produktbild** (dient als Button)
- Die **Produktkategorie**
- Den **Preis**
- Falls ein **Rabatt** vorhanden ist:
 - Den rabattierten Preis (in rot)
 - Daneben den ursprünglichen Preis
 - Die Kategorie "Sale"
- Die Kategorie "**Out of stock**" falls ausverkauft

Der User hat die Möglichkeit Produkte nach folgenden Kriterien zu **filtern und sortieren**:

- Produktkategorie
- Preis unter 10€, 25€ oder 50€
- Sortiert nach Preis aufsteigend oder absteigend
- Alphabetisch sortiert nach Produktnamen

Der User interessiert sich für ein Produkt und klickt auf ein Produktbild, er landet auf der Seite zu diesem Produkt. Dort hat er die Optionen das **Produkt in den Warenkorb** zu legen oder es zu **abonnieren**.

Hier sieht der User außerdem die Produktbeschreibung und am Ende der Seite alle **Bewertungen** in einem List View. Hier ist **Sortierung und Filterung** nach folgenden Kriterien möglich:

- Vergebene Sterne
- Sortiert nach Sternen aufsteigend oder absteigend

Angemeldet User können **Bewertungen hinterlassen**. Jede Bewertung besteht aus:

- **1-5 Sternen**
- Einem **Kommentar**

Möchte der User ein Produkt in den Warenkorb legen, kann er das auch **ohne Konto**. Er **wählt die Stückzahl** in Form von einem "-/+ "-Button aus und drückt auf "Add to cart". Besitzt das Produkt die Kategorie "Out of stock", bekommt der User einen **Fehler "Cannot add product to cart, out of stock."**

Möchte der User ein Produkt **abonnieren**, wird er aufgefordert sich **anzumelden oder zu registrieren**. Ab jetzt wird der User über **Preisveränderungen** und **Restocks** informiert. Der "**Subscribe**"-Button wird das Icon zu einem Haken geändert, wird er nochmal geklickt sind die **Benachrichtigungen wieder deabonniert**.

Der User wählt eine Stückzahl und klickt "Add to cart", im **Header** erscheint eine "1" neben dem **Warenkorb-Symbol**. Er klickt auf das Symbol und kommt auf die Warenkorb-Seite. Dort sieht der User in einem List View alle Produkte aufgezählt:

- **Produktname**
- **Preis pro Stück** zum Zeitpunkt des Hinzufügens
- **Stückzahl**
- **Gesamtpreis pro Produkt**
- Am Ende den **Gesamtpreis der Bestellung**

Auch hier besteht die Option nach den oben genannten Kriterien zu filtern und zu sortieren. In jeder Zeile befindet sich ein weiterer "-/+ "-Button zum **anpassen der Stückzahl** und ein "**Remove from cart**"-Button.

Der User passt alles zu seiner Zufriedenheit an und klickt auf "**Buy**". Spätestens hier wird der User zur Registrierung oder Anmeldung aufgefordert. Eine **Bestellung** wird erstellt und in der **Bestellhistorie** des Users gespeichert. Die Bestellhistorie ist ein List View bestehend aus:

- **Bestellnummer**
- **Gesamtpreis**
- **Datum der Bestellung**

Auch diese Liste ist **sortier- und filterbar**:

- Datum der Bestellung vor einer Woche, Monat oder Jahr

- Aufsteigend oder absteigend nach Datum sortiert

Klickt der User auf eine Bestellung, sieht er die **Rechnung**, eine einfache Auflistung ähnlich zu der im Warenkorb, nur nicht durch den User veränderbar und sie inkludiert die **User-Details**:

- **Vor- und Nachname**
- **Adresse**

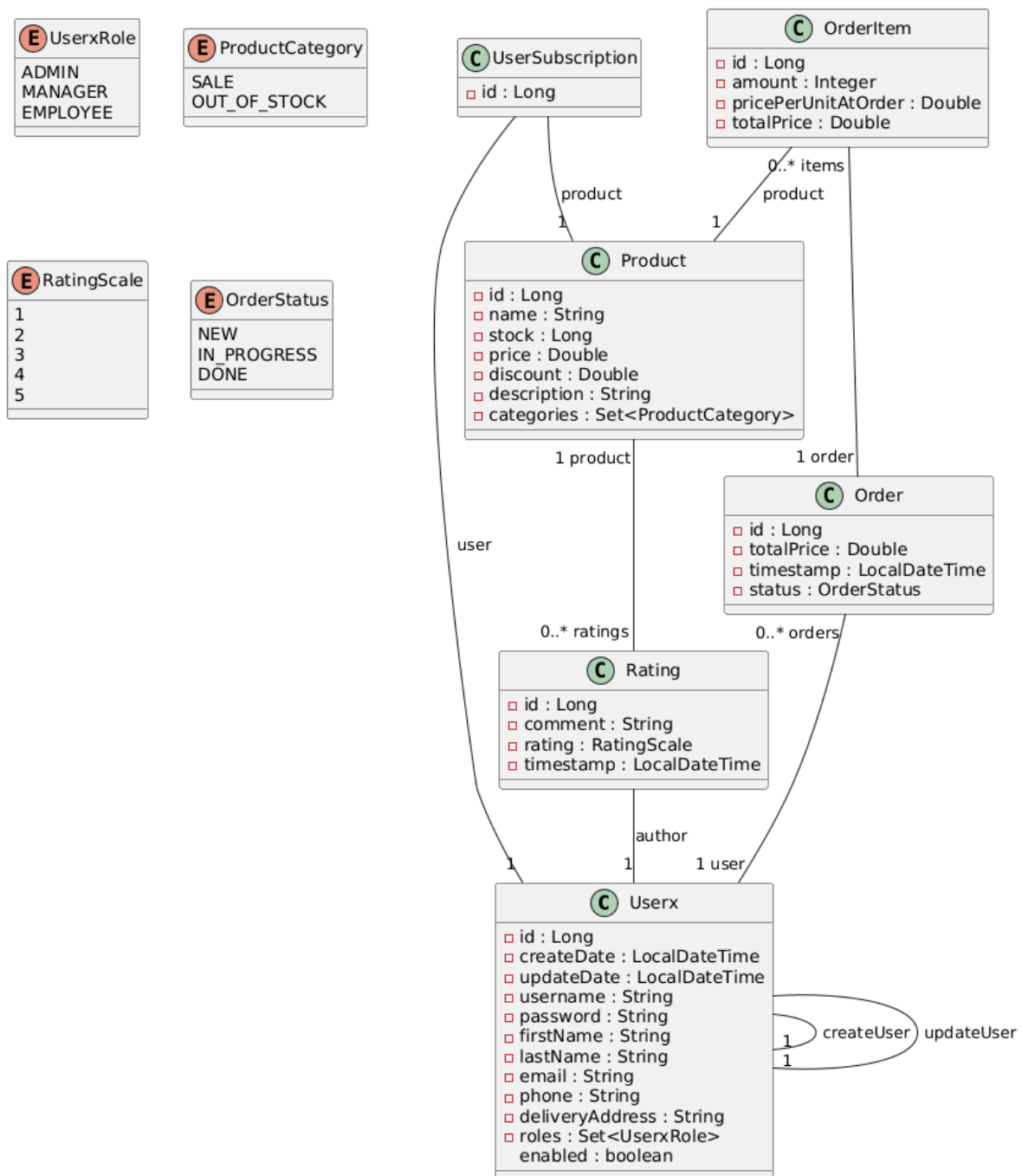
Zusätzlich zu diesen Funktionen können User mit der **Rolle**:

- **Manager**
 - Auf der Produktseite auf "Edit" klicken und Produktdetails anpassen
 - Im Header auf "Add new product" klicken und ein neues Produkt hochladen
- **Admin**
 - Auf der Produktseite auf "Edit" klicken und Produktdetails anpassen
 - Im Header auf "Add new product" klicken und ein neues Produkt hochladen
 - User-Konten verwalten

1.1.1 Sequenzdiagramm

ToDo

1.1.2 Fachliches Klassendiagramm



1.2 Meilensteine

Meilenstein 1 - Backend und Tests

Das Ziel von Meilenstein 1 ist es, die **Basisfunktionen** für die uns zugewiesenen Features zu implementieren, vorerst nur im Backend. Anschließend sollen UnitTests für das Backend geschrieben werden. Für den Warenkorb wird hier die Implementation im Frontend verlangt, da er nur dort existiert.

Deadline ist der 31.12.2026.

Meilenstein 2 - Frontend und Feature-Erweiterung

Das Ziel von Meilenstein 2 ist es, das Frontend für die uns zugewiesenen Features zu implementieren und gemeinsam die User-Verwaltung und das Abonnements-System auszuarbeiten.

Deadline ist der 12.01.2026.

Meilenstein 3 - Fertiges Produkt

Das Ziel von Meilenstein 3 ist es, das **Projekt fertig** zu stellen. Bis hierhin sollen nurnoch Inkonsistenzen behandelt, Fehler behoben und gemeinsam die Fehlerbehandlung ausgearbeitet werden.

Die **Abgabe** erfolgt dann am 02.02.2026.

Deadline ist der 30.01.2026.

1.3 Issues

Allgemein wird wie folgt nach Features aufgeteilt:

- **Produkte** (ohne Abonnements): Ljonja
- **Bewertungen**: Matteo
- **Bestellungen**: Jakob
- **Warenkorb**: Ayleen
- **User, Abonnements und Fehlerbehandlung**: Gruppenarbeit

Table 1.1: Zu erstellende Issues

Issue	Feature	Beschreibung	Zugewiesen	Deadline
Implement feature in backend	Produkte	Implement the backend for the products and test it, ratings and subscriptions excluded	Ljonja	Meilenstein 1
Implement feature in backend	Bewertungen	Implement the backend for the ratings and test it	Matteo	Meilenstein 1
Implement feature	Warenkorb	Implement the shopping cart and test it, only exists in frontend	Ayleen	Meilenstein 1
Implement feature in backend	Bestellungen	Implement the backend for the orders and test it	Jakob	Meilenstein 1
Implement feature in frontend	Produkte	Implement the frontend for the products, ratings and subscriptions excluded	Ljonja	Meilenstein 2
Implement feature in frontend	Bewertungen	Implement the frontend for the ratings	Matteo	Meilenstein 2
Implement feature in frontend	Bestellungen	Implement the frontend for the orders	Jakob	Meilenstein 2
Implement managers	User	Implement all remaining user features for managers and test them	Alle (oder Ayleen?)	Meilenstein 2
Implement subscription system	Produkte	Implement subscription logic and test it	Alle (oder Ayleen?)	Meilenstein 2
Error handling	Fehler	Collect all existing errors and implement error handling and test it	Alle	Meilenstein 3
Bug fixes	-	Finalise the project	Alle	Meilenstein 3

2 User

Registrierte User sind eine **Datenbankentität**. Der Großteil des User-Managements ist bereits implementiert.

2.1 Attribute

Jeder User hat:

- ID
- Vor- und Nachname
- Email
- Telefonnummer
- Username
- Einen Boolean-Wert `enabled`
- Einen `createUser` und `createDate`
- Einen `updateUser` und `updateDate`
- Eine Sammlung von Bestellungen, Bewertungenm Rollen und Abonnements

2.1.1 Rollen

Startet man die Web Applikation ist man vorerst ein **Besucher**, also ein unangemeldeter Nutzer, der nur die Option hat, seinen Warenkorb zu befüllen. Bei jeder anderen Funktion wird der zuerst zur Seite "Log-in or register" gebracht.

Angemeldete User haben eine oder mehrere Rollen.

Normale **User** können bestellen, die Bestellhistorie einsehen, Produkte abonnieren und Bewertungen hinterlassen.

Manager haben zusätzlich auf der Seite jedes Produkts einen "Edit"-Button, ähnlich zu dem im bereits bestehenden User-Management-System. Ihnen erscheint im Header ein "Add new product"-Button.

Admins können alles was Manager und User können, und haben Zugriff auf die User-Verwaltung.

2.2 Involvierte Klassen

- Userx: Die Datenbankentität
- UserxRole: Ein Enum zu den Rollen
 - USER
 - MANAGER
 - ADMIN
- UserxRepository: Kommuniziert direkt mit der Datenbank
- UserxService: Stellt zusätzliche Datenbankfunktionen zur Verfügung
- AuthenticatedUserService: stellt das aktuell authentifizierte User-Objekt bereit
- AuthenticationService: authentifiziert Benutzer und erzeugt nach erfolgreicher Anmeldung ein JWT zur weiteren Autorisierung
- UserxMapper: Konversion zwischen Userx und UserxDTO
- UserxCreateMapper: Konversion zwischen Userx und UserxCreateDTO
- UserxDTO: Datentransferobjekt zur User-Verwaltung
- UserxCreateDTO: Datentransferobjekt zur User-Erstellung
- LoginResponseDTO: kapselt das nach erfolgreicher Authentifizierung erzeugte JWT
- LoginRequestDTO: Datentransferobjekt zur User-Authentifizierung
- UserxController, AuthenticationController, ManagerController und AdminController: Weisen HTTP Anfragen Methoden zu

3 Produkte

Produkte sind eine **Datenbankentität**.

3.1 Attribute

Jedes Produkt hat:

- ID
- Name
- Preis
- Rabatt
- Bestand
- Produktbild
- Beschreibung
- Eine Sammlung von Bewertungen und Kategorien

3.1.1 Bewertungen

Auch Bewertungen sind eine **Datenbankentität**.

Attribute

Jede Bewertung hat:

- ID
- Autor
- Bewertung in RatingScale
- Timestamp
- Produkt

3.2 Abonnements

Abonnements sind mithilfe vom [Spring Observer](#) umgesetzt, oft auch "Publisher-Subscriber" genannt.

Bei einem Abonnement wird der User **über Restocks und Rabatte informiert**. Wir definieren also 2 **Events**, welche beide von `ApplicationEvent` erben:

- `ProductRestockEvent`
- `ProductSaleEvent`

Diese sind wie folgt aufgebaut:

```
1 public class ProductRestockEvent extends ApplicationEvent {
2     private final Long productId;
3
4     public ProductRestockEvent(Object source, Long productId) {
5         super(source);
6         this.productId = productId;
7     }
8
9     public Long getProductId() {
10        return productId;
11    }
12 }
```

Zudem brauchen wir einen **Publisher**, der über Events informiert. Der `ApplicationEventPublisher` existiert bereits in Spring und muss nur noch in einen Service injiziert werden. Das sieht dann ungefähr wie folgt aus:

```
1 @Service
2 public class ProductService {
3     @Autowired
4     private ApplicationEventPublisher publisher;
5
6     public void restockProduct(Long productId) {
7         // ...
8         publisher.publishEvent(new ProductRestockEvent(this,
9             productId));
10    }
11 }
```

Zuletzt benötigen wir noch einen Subscriber.

```
1 @Component
2 public class ProductEventListener {
3     @EventListener
4     public void handleRestock(ProductRestockEvent event) {
5         // Notify user
6     }
7 }
```

```

6      }
7
8      @EventListener
9      public void handleSale(ProductSaleEvent event) {
10         // Notify user
11     }
12 }

```

3.2.1 Ablauf

Ein Abonnement abzuschließen erfolgt mit dieser Implementation so:

1. Der User klickt im Frontend auf der Produktseite auf "Subscribe".
2. Das Frontend sendet eine Anfrage an den Backend-Endpoint, der den User für das Produkt registriert.
3. Der Backend-Service speichert die Abonnement-Relation in der Datenbank.
4. Wenn ein Event wie ProductRestockEvent oder ProductSaleEvent ausgelöst wird, empfängt der ProductEventListener dieses Event.
5. Der Listener filtert die User, die das Produkt abonniert haben, und benachrichtigt sie über die gewünschte Methode.

3.3 Involvierte Klassen

- Product, Rating und UserSubscription: Die Datenbankentitäten
- ProductCategory: Ein Enum zu den Kategorien
 - SALE
 - OUT_OF_STOCK
 - Weitere Kategorien je nach Produkttyp
- RatingScale: Ein Enum zu den vergebenen Sternen
- ProductRepository, RatingRepository und UserSubscriptionRepository: Kommuniziert direkt mit der Datenbank
- ProductService, RatingService und UserSubscriptionService: Stellt zusätzliche Datenbankfunktionen zur Verfügung
- ProductController und SubscriptionController: Weisen HTTP Anfragen Methoden zu

- ProductMapper: Konversion zwischen Product und ProductDTO
- ProductDTO: Datentransferobjekt für Frontend
- ProductCreateDTO: Datentransferobjekt zur Produkterstellung
- SubscriptionRequestDTO: Datentransferobjekt zum Abschließen eines Abonnements
- ProductEventListener: Versendet Benachrichtigungen
- ProductRestockEvent und ProductSaleEvent: Mögliche Benachrichtigungen

4 Warenkorb

Der Warenkorb ist keine Datenbankentität sondern wird per Local Storage **im Browser gespeichert**.

Beim Hinzufügen eines Produkts wird der aktuelle **Warenkorb geladen**, das **Produkt hinzugefügt** oder die **Stückzahl angepasst**, und der **Warenkorb wieder gespeichert**. Produkte können entfernt oder die Stückzahl angepasst werden. Beim Laden der Warenkorb-Seite wird der gespeicherte Warenkorb ausgelesen und angezeigt.

Das soll ungefähr so funktionieren:

```
1 // Add product to cart
2 function addToCart(productId, quantity) {
3     let cart = JSON.parse(localStorage.getItem('cart')) || [];
4     const existing = cart.find(item => item.id === productId);
5     if (existing) {
6         existing.quantity += quantity;
7     } else {
8         cart.push({ id: productId, quantity });
9     }
10    localStorage.setItem('cart', JSON.stringify(cart));
11 }
12
13 // Read cart
14 function getCart() {
15     return JSON.parse(localStorage.getItem('cart')) || [];
16 }
17
18 // Remove product
19 function removeFromCart(productId) {
20     let cart = JSON.parse(localStorage.getItem('cart')) || [];
21     cart = cart.filter(item => item.id !== productId);
22     localStorage.setItem('cart', JSON.stringify(cart));
23 }
```

4.1 Attribute

Den Warenkorb an sich gibt es nicht, nur das DTO. Jedes davon hat:

- Eine Sammlung an CartItemDTO

Jedes `CartItemDTO` hat:

- Produkt-ID
- Produktname
- Preis pro Stück
- Stückzahl

4.2 Involvierte Klassen

- `CartDTO`: Speichert Produkte und deren Stückzahl
- `CartItemDTO`: Datentransferobjekt für Produkte im Warenkorb

5 Bestellungen

Bestellungen sind **Datenbankentitäten** und für die **Zahlungsabwicklung** verantwortlich. User haben Zugriff auf ihre Bestellhistorie.

5.1 Attribute

Jede Bestellung hat:

- ID
- Gesamtpreis
- Timestamp
- Status
- Den dazugehörigen User
- Eine Sammlung von OrderItem

Jedes OrderItem hat:

- Produkt-ID
- Name
- Preis pro Stück zum Bestellzeitpunkt
- Stückzahl
- Gesamtpreis

5.1.1 Status

Der Status einer Bestellung **verändert sich automatisch**. Während die Bestellung erstellt wird ist sie NEW. Beginnt man die Zahlungsabwicklung, wird der Status auf IN_PROGRESS gestellt. Sobald die Rechnung per Email "versendet" wurde und in der Bestellhistorie einzusehen ist, ist sie unveränderbar und hat den Status DONE.

Der Status hat **keine weitere Funktion** und ist nur Aufgrund der Anforderungen implementiert.

5.2 Zahlungsabwicklung

Die Zahlungsabwicklung simuliert den Ablauf einer Bestellung. Es erfolgt **keine echte Transaktion**. Nach Abschluss wird der Bestellstatus entsprechend aktualisiert und eine Email "versendet".

5.3 Erstellung einer Bestellung

Eine Bestellung entsteht aus dem aktuellen Warenkorb des Users. Beim Abschließen des Bestellvorgangs werden die **Inhalte des Warenkorbs in eine neue Order mit entsprechenden OrderItem überführt**. Anschließend wird der Warenkorb geleert.

5.4 Involvierte Klassen

- Order und OrderItem: Datenbankentitäten
- OrderRepository: Kommuniziert direkt mit der Datenbank
- OrderService: Stellt zusätzliche Datenbankfunktionen zur Verfügung
- OrderStatus: Ein Enum zum Bestellstatus
 - NEW
 - IN_PROGRESS
 - DONE
- OrderMapper: Konversion zwischen Order und OrderDTO
- OrderDTO: Datentransferobjekt für Frontend
- OrderController: Weist HTTP Anfragen Methoden zu

6 Fehlerbehandlung